

Chapter 1

Why Becoming Agile Is Hard (But Worth It)

Many software development organizations are striving to become more agile. And who can blame them? Successful agile teams are producing higher-quality software that better meets user needs more quickly and at a lower cost than are traditional teams. Besides, who wouldn't want to be more agile? It just plain sounds good, doesn't it? It is almost as though one cannot be too thin, too rich, or too agile. But beyond the buzzword and hype, organizations that take becoming agile seriously by adopting a process such as Scrum are seeing dramatic benefits.

They are seeing significant gains in productivity with corresponding decreases in cost. They are able to bring products to market much faster and with a greater degree of customer satisfaction. They are experiencing greater visibility into the development process, leading to greater predictability. And for them, out-of-control, will-it-ever-be-done projects have become a thing of the past.

One company to realize these benefits by adopting Scrum is Salesforce.com. Founded in 1999 in a San Francisco apartment, Salesforce.com is one of the true, lasting dot-com-era success stories. With revenue of more than \$450 million and 2,000 employees in 2006, Salesforce.com had noticed the frequency of its releases had dwindled from four a year to one a year. Customers were getting less and waiting longer to get it; something needed to be done. The company decided to transition to Scrum. During the first year of making the switch, Salesforce.com released 94% more features, delivered 38% more features per developer, and delivered over 500% more value to its customers compared to the previous year (Greene and Fry 2008). In the ensuing two years, revenue more than doubled to more than \$1 billion. With results like these, it is not surprising that so many organizations have transitioned to Scrum. Or at least tried to.

I say "tried to" because transitioning to Scrum and other agile methods is hard—much harder than many companies anticipate. The changes required to reap all of the rewards being agile can bring are far reaching. These changes demand a great deal from not only the developers but the rest of the organization as well. Changing practices is one thing; changing minds is quite another. It is my aim in this book to show not only how to transition well but also how to succeed long term.

I’ve personally witnessed several failed agile adoptions that could have been prevented. The first was in a company that had spent more than a million dollars on its transition effort. Executives brought in outside trainers and coaches and hired five people into an “Agile Office” to which new Scrum teams could turn for advice. The company’s failure was the result of thinking that the implications of adopting Scrum would be restricted to only the development organization. The executives who initiated this transition thought that educating and supporting developers would be sufficient. They failed to consider how Scrum would touch the work of salespeople, the marketing group, and even the finance department. Without changes to these areas, organizational gravity pulled the company back where it had started.

For completely different reasons, Josef ultimately failed at introducing Scrum to his company. A newly promoted and first-time project manager, Josef was instantly attracted to Scrum because it fit his natural management style. Josef easily convinced his team—who had all been his peers as little as one month before—to try Scrum on their new project. The project was wildly successful, earning accolades for the team and winning Josef the chance at a much larger project. Josef introduced the new project team to Scrum, and most members were willing to try the new approach. Although those working on the project were happy to use Scrum, some of the functional managers to whom they reported got nervous about what Scrum might mean to their careers. Josef’s luck ran out. The functional managers—in particular the directors of quality assurance and database development—banded together and convinced the vice president of engineering that Scrum was inappropriate for projects of the complexity and importance being done in their company.

Caroline fared a little better. A vice president of development in a large data management company, Caroline had more than 200 developers in her organization. After seeing the benefits of Scrum on one project, she excitedly launched an initiative to introduce Scrum across her division. All employees were provided with training or coaching. Within a few months nearly all teams were producing working software at the end of each two-week sprint. This was great progress. When I visited this company a year later, though, the employees had failed to make any additional headway. To be sure, teams were producing higher-quality software and doing it a bit faster than they had before starting with Scrum, but her company’s gains were only a fraction of what they could have been. Caroline’s company had forgotten that continuous improvement is part of Scrum.

Frightening, isn’t it? Each of these failures was a well-intentioned effort to transition to Scrum. Yet all the good intentions in the world could not keep them from failing. Don’t worry, though. Transitioning to Scrum may be hard, but it’s entirely possible with the right approach. In this chapter we examine why transitioning to any agile development process, including Scrum, is especially difficult.

We detail some of the challenges that derailed the companies I've mentioned. Most important, though, we look at the reasons why the benefits of becoming an agile organization are more than worth the effort.

Why Transitioning Is Hard

All change is hard. I've seen employees in an uproar over something so small as a change in their company's healthcare plan. Larger changes can be even more painful. But there are certain attributes of transitioning to Scrum that make it more difficult than most other changes. They are as follows:

- Successful change is not entirely top-down or bottom-up.
- The end state is unpredictable.
- Scrum is pervasive.
- Scrum is dramatically different.
- Change is coming more quickly than ever before.
- Best practices are dangerous.

Successful Change Is Not Entirely Top-Down or Bottom-Up

Successful organizational change cannot be fully top-down or bottom-up. In a top-down change, a powerful leader shares a vision of the future and the organization follows the leader toward that vision. Imagine a charismatic, respected, and powerful leader such as Steve Jobs telling his Apple employees that they are moving beyond computer hardware and software to dominate digital music. His reputation and style might have pointed the company in a new direction, but that alone would not have been enough to pull off such a monumental feat. Change management expert John Kotter agrees.

No one individual, even a monarch-like CEO, is ever able to develop the right vision, communicate it to large numbers of people, eliminate all the key obstacles, generate short-term wins, lead and manage dozens of change projects, and anchor new approaches deep in the organization's culture. (1996, 51–52)

By contrast, in a bottom-up change, a team or some individuals decide that a change is needed and they set about making it happen. Some teams undertake a bottom-up change with an “ask for forgiveness later” attitude. Others flaunt that they are breaking the rules. Still others attempt to fly under the corporate radar as long as possible.

Most successful changes, and especially a change to an agile process like Scrum, must include elements of both top-down and bottom-up change. Mary

Lynn Manns and Linda Rising agree, writing in *Fearless Change*, “We believe that change is best introduced bottom-up with support at appropriate points from management—both local and at a higher level” (2004, 7). An organization attempting to transition to Scrum without support from the top will encounter resistance that cannot be overcome from below. This usually occurs as soon as the new Scrum process begins to affect how areas outside the original team do their work. In response, middle managers protect their departments by striking out against changes created by Scrum. Top-down support will be needed to remove these kinds of impediments and obstacles.

Similarly, without bottom-up engagement, the transition will feel like sitting under a ceiling fan in an open-air restaurant in Mexico: just a bunch of hot air blowing down from above. When this happens, individuals resist being told what to do. Bottom-up participation will be needed because it will be the individual team members who work through the issues of discovering how Scrum will work best within their organization.

Key to any successful adoption of Scrum will be combining elements of both bottom-up and top-down change.

The End State Is Unpredictable

Perhaps you’ve read a book on Extreme Programming and have decided that is the right approach for your company. Or maybe you attended a Certified Scrum-Master training course and think Scrum sounds good. Or maybe you read a book on a different agile process, and it sounds perfect for your organization.

In all likelihood, you’re wrong.

None of these processes as described by their originators is perfect for your organization. Any may be a good starting point, but you will need to tailor the process to more precisely fit the unique circumstances of your organization, individuals, and industry. Alistair Cockburn concurs: “Having a chance to change or personalize a process to fit themselves seems to be a critical success factor for a team to adopt a process. It’s the act of creation that seems to bind teams to ‘their own’ process.”¹

You may have a clear vision of what “doing Scrum” means to you, and you may get others to buy into exactly that same vision, but where the organization ends up is likely to be somewhat different. In fact, to even refer to end states in a Scrum transition is incorrect; there can be no end state in a process that calls for continuous improvement.

This creates a problem for an organization that wants to transition to Scrum through a traditional change approach that relies on gap analysis and then on closing the identified gaps. If we cannot anticipate the end state of a Scrum

¹ This and all other uncited references are personal communications between the speaker and me.

transition, we cannot identify all of the gaps between there and the current state. So, a gap analysis–driven change approach will not work. The closest we can come is to identify gaps between where we are now and an improved, intermediate state.

After identifying these smaller gaps, though, we are still left with the problem of how to close them. It is difficult (and often impossible) to predict exactly how people will respond to the many small changes that will be needed on the way to becoming agile. Teamwork expert Christopher Avery views organizations as living systems.

We can never direct a living system, only disturb it and wait to see the response.... We can't know all the forces shaping an organization we wish to change, so all we can do is provoke the system in some way by experimenting with a force we think might have some impact, then watch to see what happens. (2005, 22–23)

So, a transition to Scrum cannot be a process that “articulates and defines the entire change process required to bridge the gap between ‘as is’ and ‘to be’ and creates tactical plans,” as I read in a traditional change management book (Carr, Hard, and Trahant 1996, 144–5). Creating such a plan would require leaping two impossible hurdles: first, knowing exactly where we’ll want to end up; and second, knowing exactly the steps to get there. Because we cannot overcome these impossibilities, the best we can do is adopt a “provoke and observe” approach (Avery 2005, 23) in which we try something, see if it moves us closer to an intermediate, improved state, and if so do more of it. These pokings and proddings of the organization are not random. They are carefully selected based on experience, wisdom, and intuition to drive a successful transition to Scrum.

SEE ALSO

Chapter 4, “Iterating Toward Agility,” will describe the overall process I recommend when adopting Scrum.

Scrum Is Pervasive

When a change is isolated, when it doesn’t affect everything a person does, that change is often easier to introduce into an organization. Consider the case of an organization using a non-agile process that decides to introduce a mandatory operational readiness review before an application is deployed onto the company’s web servers. This is a relatively isolated change. Sure, there will be some developers who will hate the new procedure and will complain, perhaps loudly. But, when it comes down to it, this is not a pervasive change. Even if they don’t like this change, they can still continue doing the majority of their work unscathed.

Consider now the case of a developer transitioning to Scrum. This developer has to work on smaller pieces of work at a time to complete something by the end of each timeboxed sprint. The developer might have to write automated tests to go with each new bit of code. She might even alternate short bouts of testing and coding in something called test-driven development. And she might need to do all this with her headphones off while pair programming. These are fundamental

changes. They aren't something relegated to a few hours a day or week, as code inspections might be. This type of fundamental change is difficult because it pervades everything about a developer's workday. Resistance will be greater because the impact is greater.

Adopting Scrum is pervasive in a second way as well. Being agile will have implications to the organization that reach far outside the software development department. Introducing the operational readiness review would almost certainly not impact finance, sales, or other departments. But each of those departments can be impacted by Scrum. Finance groups will have to reconcile company policies on capitalizing or expensing with the way Scrum projects run. Sales will want to consider altering how they communicate date and scope commitments and may change how they structure contracts. With more groups affected by a move to Scrum, there is more chance for resistance and certainly more chance for misunderstandings. These add up to make transitioning to Scrum harder than other changes.

SEE ALSO

The impact of Scrum on other groups, such as finance, operations, human resources, and others is discussed in Chapter 20, "Human Resources, Facilities, and the PMO."

Scrum Is Dramatically Different

Not only do the changes created by adopting Scrum pervade everything development team members do, but also many of the changes go against much of their past training. Many testers, for example, have learned that their job is testing for compliance to a specification. Programmers have been trained that a problem is to be analyzed in depth and a perfect solution designed before any coding begins. On a Scrum project, testers and programmers need to unlearn these behaviors. Testers learn that testing is also about conformance with user needs. Programmers learn that a fully considered design is not always necessary (and sometimes not even desirable) before coding begins. Abby Fichtner, who shares her thoughts on her Hacker Chick blog, has told me she agrees with how hard this adjustment can be for programmers.

Getting used to emergent design is hard because it feels like you're going to be just hacking! And if you've prided yourself on being a very good developer and always doing well-thought-out designs, it turns your whole world upside down and says "no, all those things you thought made you great, now those same things actually make you a bad developer." Very world-rocking stuff.

Because transitioning to Scrum involves asking people to work in ways that are unfamiliar and run counter to training and experience, people are often hesitant, if not outright resistant, to the change. Consider, for example, the case of Terry, a senior and respected programmer in his company. Terry had participated in a hands-on full-day class on test-driven development and was convinced of its benefits. An enthusiastic Terry returned to the office expecting to stop doing big,

SEE ALSO

Emergent design and test-driven development are discussed in Chapter 9, "Technical Practices."

up-front designs and allow design to emerge through the use of test-driven development. It didn't go as smoothly as he thought it would. He wrote me an e-mail describing his deflating experience.

Getting the other programmers to even try test-driven development was much harder than I thought. I tried pushing it as a way to skip the long up-front design phases we'd become accustomed to, but failed miserably. After a few months I got the other developers to start writing tests first, but only because it was a good idea on its own. They still wouldn't abandon the lengthy up-front design phase. It took me another year to make much progress shortening that, and we could still go much shorter.

Change Is Coming More Quickly Than Ever Before

Back in 1970 Alvin Toffler coined the term *future shock*, saying that it is the disorientation people feel when confronted with “too much change in too short a period of time” (1970, 4). Human, and therefore organizational, capacity to change is limited—ask people to change too many things at the same time and they shut down; the shattering stress and disorientation of future shock kicks in.

In many organizations, employees have been suffering from future shock for years. Teams are asked to do more with fewer people. Outsourcing and distributed teams have become increasingly common. These adjustments were preceded by the rush to move applications to a client/server model, then onto the web, and then into services. Add to these the constant, and constantly accelerating, rate of change in technology itself—new languages, new tools, new platforms—and future shock is now. It should not be surprising that transitioning to Scrum can often be the change that pushes people into future shock. The pervasive nature of adopting Scrum and the fundamental changes it causes in how people work and interact have a higher risk of triggering the future shock effect.

Best Practices Are Dangerous

With most organizational change, after someone figures out the right or best way to do something, that way of doing it is captured as a “best practice” and shared with everyone else. For some types of work, collecting and reusing best practices is a tremendous aid to the change effort. An organization that is selling a product to a new type of customer may, for example, capture best practices for overcoming objections from potential customers. When transitioning to Scrum, however, collecting best practices can be dangerous.

Like sirens singing to us from the rocks, best practices tempt us to relax and stop the effort of continuous improvement that is essential to Scrum. Taiichi Ohno, originator of the Toyota Production System, has written that “there is something

called standard work, but standards should be changed constantly. Instead, if you think of the standard as the best you can do, it's all over." Ohno goes on to say that if we establish something as the "best possible way, the motivation for kaizen [continuous incremental improvement] will be gone" (1982).

Although team members should always look to share with one another their newly discovered good ways of working, they should resist the urge to codify them into a set of best practices. One example of a best practice gone awry is the company that decided that all daily scrums needed to be held no later than 10:00 a.m. I found this an extremely unnecessary dictate. I'm not entirely sure what purpose the dictate served. But many employees took the rule to be further proof that "Scrum is all about micro-management."

THINGS TO TRY NOW

- ❑ Think about your current transition to Scrum. Are you just getting started, in the middle, or feeling like you're nearing the end of the transition push? No matter where you are, identify the primary obstacle you think may be holding you back from the next level of success.

Why It's Worth the Effort

Despite all the reasons why transitioning to Scrum can be particularly difficult, stakeholders in companies that have made the transition are happy they've done so. One reason stakeholders are so satisfied is that time-to-market is reduced when using an agile process like Scrum. This faster time-to-market is enabled by the higher productivity of agile teams, which is in turn the result of the higher quality seen on agile projects. Because employees are freed up to do high-quality work and because they see their work delivered sooner into the hands of waiting users, job satisfaction goes up. With higher job satisfaction comes more engaged employees, which leads to more productivity gains, initiating a virtuous cycle of continued improvement.

The rest of this chapter looks in more depth at these claims. In doing so I present evidence in support of each. Some of the evidence is anecdotal and drawn from my experience, experiences of my clients and colleagues, or experiences reported in magazines or at conferences. Additionally, though, the claims are supported by data from the following sources:

- A rigorous comparison of 26 agile projects against a baseline database of 7,500 primarily traditional development projects. This study was conducted by Michael Mah, managing partner of QSM Associates (QSMA), which has been collecting productivity, quality, and other metrics on projects for more than 15 years. The agile projects Mah studied ranged in size from 60 to 1,000 people (Mah 2008).

- Various academic and research papers, including aggregate research by David Rico, Ph.D., who surveyed 51 published studies of agile projects (2008).
- An online survey of more than 3,000 people conducted by agile tool vendor, VersionOne (2008), and another of 642 people conducted by *Dr. Dobb's Journal* (Ambler 2008a), a popular development magazine. Each survey was conducted in 2008. Industry surveys such as these cannot, of course, be taken as definitive. Individuals opting to take such surveys are probably predisposed toward favorable views of agile. Results from these surveys are presented because they are more representative than conclusive. These surveys will be referenced as VersionOne and DDJ in the sections that follow.

In the following sections we look at these reasons why transitioning to an agile process like Scrum is worthwhile:

- Higher productivity and lower costs
- Improved employee engagement and job satisfaction
- Faster time to market
- Higher quality
- Improved stakeholder satisfaction
- What we've been doing no longer works

Higher Productivity and Lower Costs

There is unfortunately no universally agreed-upon measure of productivity. Martin Fowler has gone so far as to say that measuring productivity of developers is impossible (2003). And although I agree with Fowler, I do think it is possible to measure proxies or stand-ins for productivity. Some teams use the number of lines of code as a proxy for productivity. Others use as a proxy the number of function points delivered or simply the number of features delivered, ignoring that not all features are the same size. Are there problems with these proxies? Absolutely. But I think the usefulness of proxy productivity measures is justified if we can reasonably make the assumption that data has not been gamed by teams fabricating lines of code or function points by duplicating code, failing to take advantage of reuse, or so on. In many cases, especially those involving large data sets as the QSMA study does, I think this is a reasonable assumption.

QSMA calculates a productivity index for the projects in its database. This index takes into consideration effort, schedule, technical difficulty, and more and is an attempt to help make cross-team comparisons more meaningful. In his comparison between agile and traditional projects, Mah found agile projects to be 16%

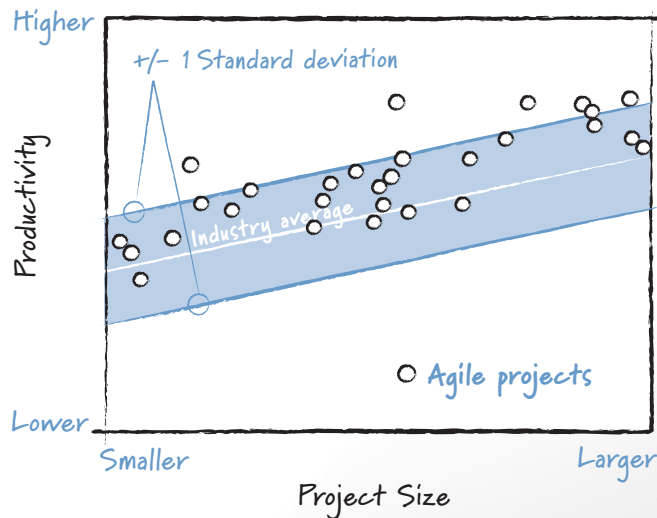
SEE ALSO

Data from this chapter is summarized in Microsoft PowerPoint and Apple Keynote presentations available at www.succeeding-withagile.com.

more productive, an increase that he found to be statistically significant. Figure 1.1 shows the agile projects (as dots) compared to the average productivity and one standard deviation around it in the QSMA database. As you can see, most of the dots are above the industry average line, with a handful of projects more than one standard deviation more productive than the industry average.

FIGURE 1.1

Agile teams are significantly more productive than the industry average.
Source: Mah 2008.



The QSMA results are corroborated by both the DDJ and VersionOne surveys. Eighty-two percent of participants in the DDJ survey felt that productivity was somewhat or much higher when using agile methods like Scrum than it was before. Only 5% felt productivity was somewhat or much lower. Seventy-three percent of the VersionOne respondents believed that being agile had significantly improved (23%) or improved (50%) productivity.

It stands to reason that if people are productive, costs will be lower. The VersionOne and DDJ studies both bear this out, as can be seen in Table 1.1.²

David Rico's survey of case studies of agile teams published through 2008 is shown in Table 1.2. Rico found that the median reported productivity increase was 88% and the median cost savings was 26%. These indicate solid evidence that agile teams are more productive, which leads to cost savings to their projects.

² The VersionOne survey asked respondents to answer on a scale that included Significantly Improved, Improved, No Benefit, Worse, and Much Worse. The DDJ survey used a similar scale but used Much Higher, Somewhat Higher, No Change, Somewhat Lower, and Much Lower. For improved readability, all tables in this chapter use the labels from the VersionOne survey.

Development Cost	DDJ	VersionOne
Improved	32%	30%
Significantly Improved	5%	8%

TABLE 1.1

A significant number of survey respondents report that agile improved development costs.

As encouraging as these numbers are, they tell only part of the story. A significant benefit to being agile—but one not reflected here—is that agile teams are less likely to build functionality that is no longer needed. A common criticism of a sequential development process is that by the time the software is delivered, the users no longer need the functionality being provided. Because of the frequent feedback, timeboxed sprints, and ability to reprioritize each sprint, a Scrum team is more likely to work only on features users really need. Were we to include this in our measurement of productivity, we would see even more dramatic results.

Category	Lowest Reported Improvement	Median Improvement	Highest Reported Improvement
Productivity	14%	88%	384%
Cost	10%	26%	70%

TABLE 1.2

Impact of agile on productivity and cost. Source: Rico 2008.

Improved Employee Engagement and Job Satisfaction

One factor contributing to the higher productivity and lower costs on agile projects may be that employees enjoy their jobs more. Fifteen months after adopting Scrum, Salesforce.com surveyed its employees and found that 86% were having a “good time” or the “best time” working at the company. Prior to adopting Scrum, only 40% said the same thing. Further, 92% of employees said they would recommend an agile approach to others. Results such as these are common; many of my clients have done employee satisfaction surveys and always with similar results. In its industrywide survey, VersionOne found that 74% of those surveyed reported morale was improved (44%) or significantly improved (30%).

One reason why employees may enjoy their jobs more is because of the sustainable pace promoted by agile processes. Chris Mann and Frank Maurer of the University of Calgary studied the amount of overtime worked by one team for the year before becoming agile and the first year after (2005). They found that before implementing agile practices, team members worked an average of 19% overtime. After adopting an agile process, that dropped by nearly two-thirds to an average of 7% overtime. Further, even though overtime was occasionally needed even after adopting agile practices, there was less variability in the amount

required, as measured by the standard deviations of the team before and after moving to agile. Johannes Brodwall, an agile software architect, says, “Overtime seems to be much less common after we started with agile. Testers are especially noticing the effect. They used to have extremely chunky workloads.”

A lack of overtime is likely just one factor contributing to higher job satisfaction among people working on agile teams. There are also the benefits of having more control over your day-to-day work, seeing the results of your work get used sooner, working more closely with coworkers, creating products that are more likely to meet customer and user expectations, and so on. Employees who are happier with their jobs and with their employers will be more engaged in the work they do. Greater employee engagement will result in numerous benefits to the organization.

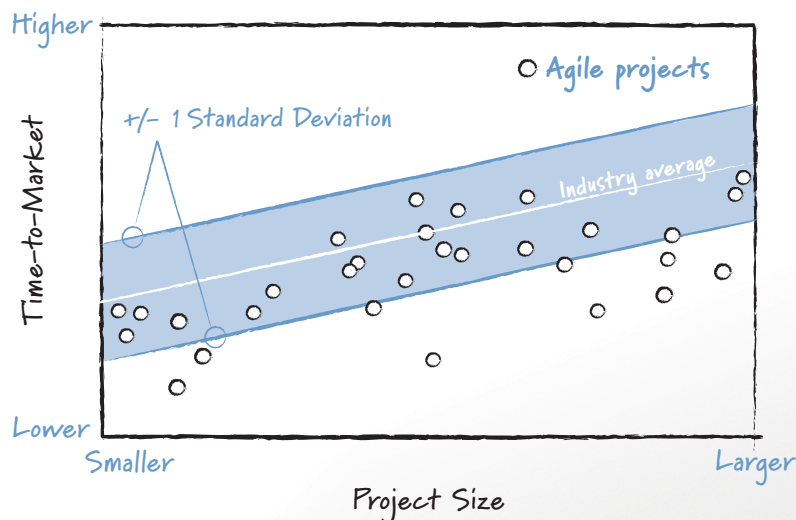
Faster Time to Market

Agile teams tend to release their products more quickly than do traditional teams. According to the VersionOne study, 64% of participants report that time to market has been improved (41%) or significantly improved (23%). The QSMA study comparing 26 agile projects to a database of 7,500 mostly traditional projects found that agile projects have a 37% faster time to market, as shown in Figure 1.2.

Agile teams have faster times to market for two reasons. First, the higher productivity of an agile team allows them to produce functionality more quickly. Second, agile teams are more likely to release incrementally. When stakeholders realize that a team can produce valuable functionality every sprint, they often decide that they do not need to wait for one big-bang delivery of all functionality.

FIGURE 1.2

Agile projects have a 37% faster time to market compared to the industry average. Source: Mah 2008.



Salesforce.com noticed the benefit of this immediately after its rapid transition to Scrum (Greene and Fry 2008). Figure 1.3 shows the cumulative number of features delivered to customers in 2006 (before adopting Scrum) and 2007 (after initiating the transition around the start of the year). This figure shows a simple metric: the raw number of features delivered and when they were delivered and a powerful view of the additional value provided to customers in the first year of using Scrum.

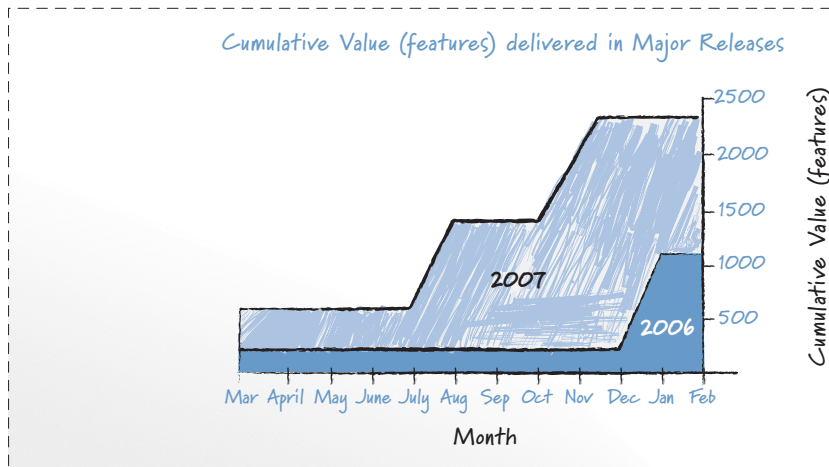


FIGURE 1.3

The cumulative value of features delivered by Salesforce.com in 2006 (pre-Scrum) and 2007 (Scrum).

Higher Quality

If you ask a Scrum team what enables them to be more productive than in their pre-Scrum days, most will say that at least part of their success is that they are consistently producing higher-quality work. Without bugs left behind to drag the team down, they can move quickly and consistently forward. Quality is improved because working at a sustainable pace prevents sloppiness. Quality is also improved through many of the engineering practices such as pair programming, refactoring, and a strong emphasis on early and automated testing.

David Rico's research bears out the claim that agile teams produce higher-quality products. In his survey of 51 published studies of agile projects, he found a minimum quality improvement of 10% and a median improvement of 63%. Rico's research matches my experience at clients where I've been able to measure and report on quality. For example, ePlanServices provides retirement plans to medium-sized businesses. The service is provided largely through a powerful web application. In the first nine months after initiating a Scrum transition, their defect rate per thousand lines of code dropped by 70%.

The VersionOne survey also bears out the claim for higher quality with agile processes such as Scrum. Sixty-eight percent of participants answered that agile had improved (44%) or significantly improved (24%) software quality. Further,

84% of respondents felt that agile had reduced the number of software defects by 10% or more; 30% felt agile had reduced the number of defects by 25% or more. The DDJ survey reported similar results, with 48% saying quality was somewhat higher and 29% saying it was much higher.

Improved Stakeholder Satisfaction

Given all of the benefits of agile processes thus far, it is not surprising that they lead to improved stakeholder satisfaction. The DDJ survey found that 78% of survey participants believe that using an agile process has led to somewhat higher (47%) or much higher (31%) stakeholder satisfaction.

One reason stakeholders are more satisfied by agile processes is because their practices are more friendly toward the shifting priorities that are a fact of life in today's fast-paced, competitive organizations. In the VersionOne study, 92% of participants felt that agile improved the ability to manage changing priorities. Additionally, along with gaining the ability to more easily change priorities, stakeholders on agile projects learn the impact of change. A stakeholder at PetroSleuth, a small development company in the oil and gas industry, found that to be true.

The Scrum process has led to our being more involved in the daily review and discussion. This has led to us being more aware, and being held accountable earlier in the process for any changes.
(Mann and Maurer 2005, 77)

The VersionOne survey looked deeper into additional factors leading to stakeholder satisfaction. Table 1.3 shows the high percentages of survey participants who reported that agile leads to better alignment between the technology and business groups, reduced project risk, better ability to manage changing priorities, and improved project visibility. Steve Fisher, a senior vice president at Salesforce.com and stakeholder to many of the agile teams there, says adopting Scrum has “delivered total visibility, total transparency, and unbelievable productivity...a complete win” (Greene 2008).

TABLE 1.3

Some of the reasons stakeholders are satisfied with agile.

	Improved	Significantly Improved
Enhanced ability to manage changing priorities	41%	51%
Improved project visibility	42%	41%
Improved alignment between IT and business goals	39%	27%
Reduced project risk	48%	17%

What We've Been Doing No Longer Works

One final reason to consider changing to Scrum is if your current development process is no longer working. When a process that has worked in the past stops working, a common tendency is to do more of it. This was certainly the case at Yahoo!, where chief product officer Pete Deemer was one of the first to recognize the need for change.

Originally, Yahoo! tried Scrum purely out of desperation—the waterfall approach was clearly not working—and a year-long attempt to do the waterfall “better” through more thorough planning and analysis, more in-depth documents, more sign-offs, and so on was making things worse, not better. For the teams that saw benefits, which were most of the teams that tried Scrum, the benefits were visible almost immediately.

Clinton Keith, former chief technology officer at High Moon Studios, developer of console-based video games, tells a similar story.

As successful project managers at a well-funded startup, we felt we could “apply more waterfall” to our ambitious new projects. This had the opposite effect of what we hoped for and the projects spiraled out of control. Our assumptions were wrong and forced us to rethink how we were managing projects.

- ❑ Identify the benefits you have gained from using Scrum so far.
- ❑ If you have not yet gathered metrics on quality, employee morale, stakeholder satisfaction, or so on, select a few factors of interest and measure a baseline you can compare against later.
- ❑ If you gathered baseline measurements earlier and have been doing Scrum for at least three or six months, remeasure and see what progress has been made. Create your own “why Scrum is worth it” charts that you can share with other teams as they begin to transition to Scrum or with existing teams who are having difficulty sticking with it.

**THINGS TO
TRY NOW**

Looking Forward

Becoming agile is hard. It is harder than most other organizational change efforts I've witnessed or been part of. I started this chapter by laying out some of the reasons why this is so, including the need to change from the top-down and bottom-up simultaneously, the impossibility of knowing exactly what the end state will look like, the dramatic and pervasive changes caused by Scrum, the difficulty of